

MIIA: Visión Artificial, PEC 1

Alejandro de Cora

2026-11-12

Tabla de contenidos

MIIA: Visión Artificial, PEC 1 3

1 Eliminación de ruido 4
- 1.1 Ejercicio 1.1 . . . 4
- 1.2 Ejercicio 1.2 . . . 4

2 Transformaciones geométricas 7
- 2.1 Ejercicio 2.1 . . . 7
- 2.2 Ejercicio 2.2 . . . 8
  - 2.2.1 Implementación de las transformaciones de forma individual . . . 8
  - 2.2.2 Implementación de las transformaciones compuestas . . . 8
  - 2.2.3 Transformar la región de interés (ROI) . . . 10
- 2.3 Ejercicio 2.3 . . . 10
  - 2.3.1 Deformación de una imagen . . . 10
  - 2.3.2 Transformación no lineal . . . 11

3 Calibración de la cámara 12
- 3.1 Ejercicio 3.1 . . . 12
  - 3.1.1 Evaluación del error de calibración . . . 13
- 3.2 Ejercicio 3.2 . . . 14

4 Flujo óptico y seguimiento 17
- 4.1 Ejercicio 4.1 . . . 17
  - 4.1.1 Lucas-kanade optical flow . . . 17
  - 4.1.2 Dense optical flow . . . 17
- 4.2 Ejercicio 4.2 . . . 19

5 Segmentación sin conocimiento del dominio 22
- 5.1 Umbralización . . . 22
  - 5.1.1 Umbralización global . . . 22
  - 5.1.2 Umbralización local . . . 22
- 5.2 K-means clustering . . . 27
- 5.3 Segmetación con MeanShift . . . 30
- 5.4 Segmentación Watershed . . . 32

Referencias 34

MIIA: Visión Artificial, PEC 1

Prueba de evaluación continua 1.

El material fuente está escrito principalmente como notebooks y se renderiza con Quarto [1].

1 Eliminación de ruido

1.1. Ejercicio 1.1

La aplicación **image-playground** para realizar procesamiento de imágenes se encuentra desplegada en huggingface [2]: https://huggingface.co/spaces/vichel0creg0/image-playground.

También puede ejecutarse de forma local utilizando el comando:

```
# Lanza la interfaz UI de la aplicación image-playground de forma local
!uv run miia-pec1 ui --port 5432
```

1.2. Ejercicio 1.2

Para reducir el ruido, se ha diseñado un pipeline de cuatro etapas.

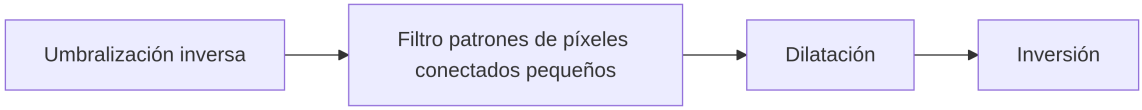
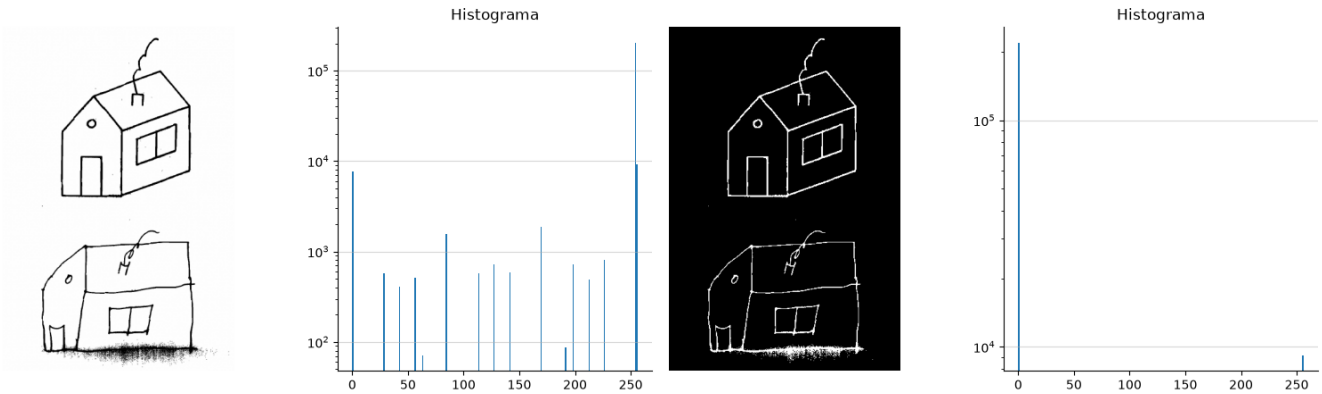


Figura 1.1: Pipeline de eliminación de ruido

En la Figura 1.2 se muestra la primera etapa del pipeline, consiste en una umbralización binaria inversa. Esta operación permite eliminar tonalidades de grin intermedias y obtener una representación binaria en la que los trazos de los dibujos aparecen en blanco y el fondo en negro.



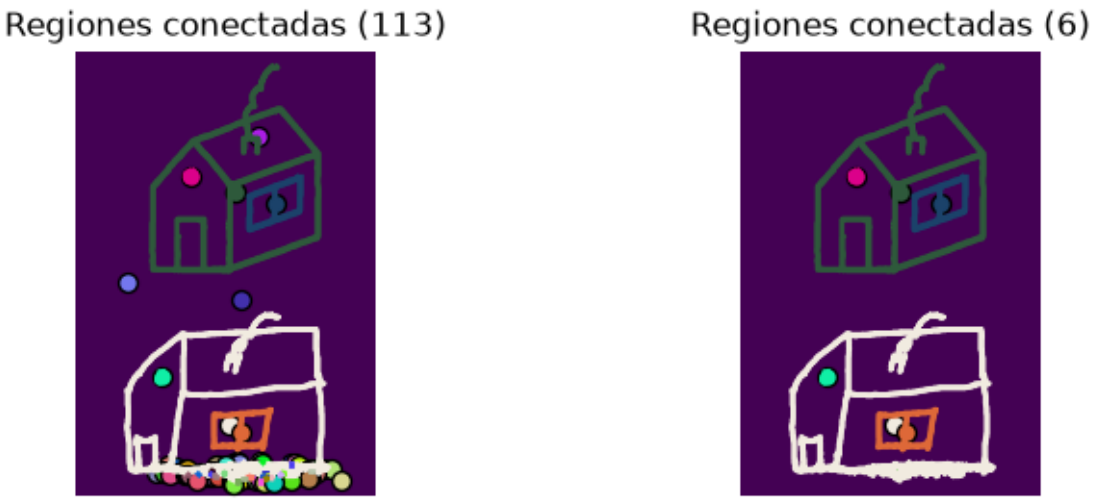
(a) Imagen original y su histograma

(b) Imagen binarizada y su histograma

Figura 1.2: Primer paso del pipeline: Umbralización inversa

Para loa segunda etapa del pipeline se utiliza el procedimiento propuesto por Moracho [3], que se basa en el filtrado de los componentes conectados según su área.

En la Figura 1.3 se muestra como el número de componentes conectados se reduce de 83 a 9, eliminando gran parte de las pequeñas regiones asociadas al ruido.



(a) Antes del filtrado

(b) Después del filtrado

Figura 1.3: Componentes conectadas

Finalmente, Figura 1.4, se aplica una dilatación morfológica para aumentar ligeramente el grosor de la los trazos y reforzar las líneas que hayan quedado demasiado finas tras las etapas anteriores.

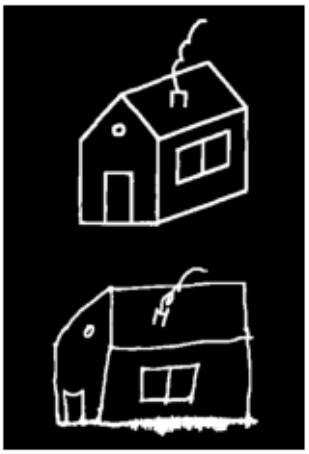


Figura 1.4: Dilatación morfológica, engrosamiento de los trazos de la imagen

En la Figura 1.5, se presenta una comparación de las imágenes de entrada con los resultados obtenidos en cada etapa de pipeline.

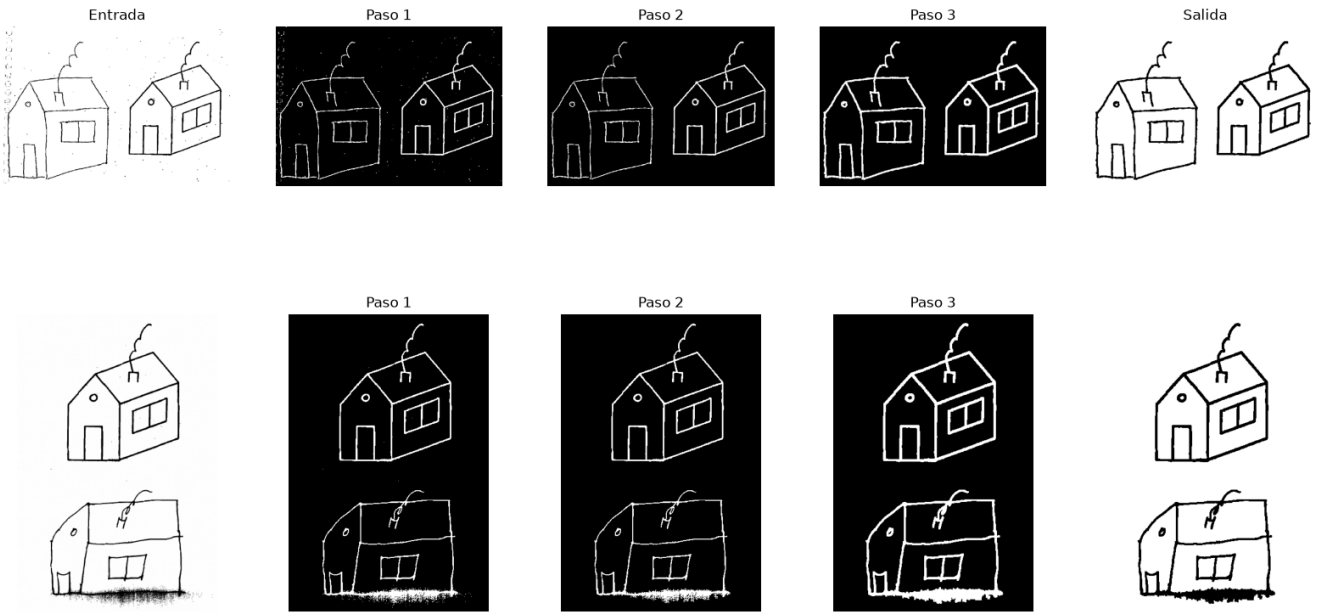


Figura 1.5: Estapas del pipeline de filtrado de ruído

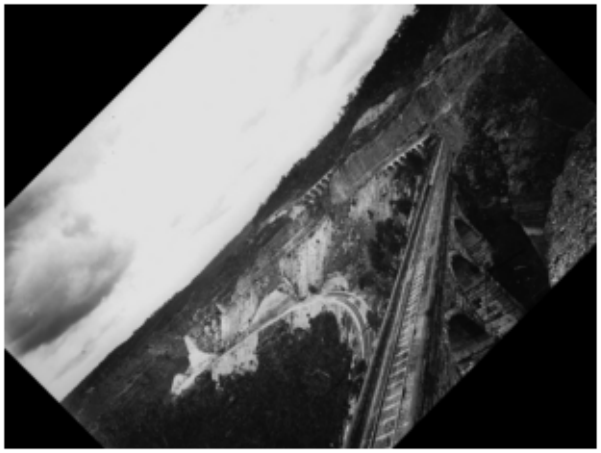
2 Transformaciones geométricas

2.1. Ejercicio 2.1

Al rotar una imagen las dimensiones del marco de salida cambian, si no ajustamos las dimensiones al nuevo marco perdemos información de la imagen original.



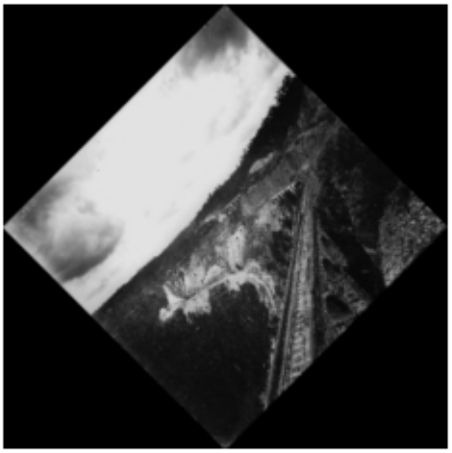
(a) Imagen original



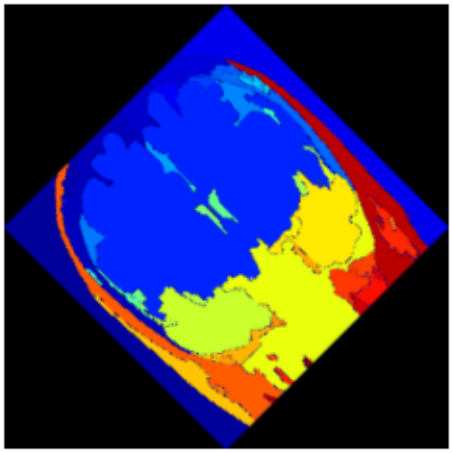
(b) Imagen rotada 45º en el frame orginal

Figura 2.1: Rotación

Para evitar la pérdida de información durante la rotación de una imagen, es necesario ajustar las dimensiones del marco de salida de modo que contenga completamente la imagen transformada [4].



(a) Imagen zigzag



(b) Imagen brainLabels

Figura 2.2: Escalado y rotación de imágenes con ajuste del marco de salida

2.2. Ejercicio 2.2

2.2.1. Implementación de las transformaciones de forma individual

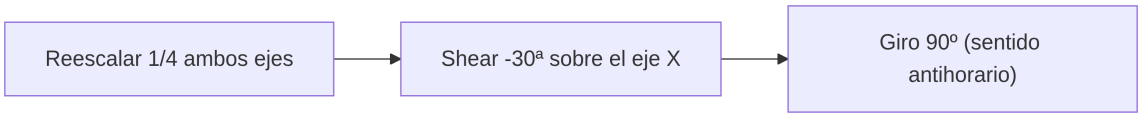


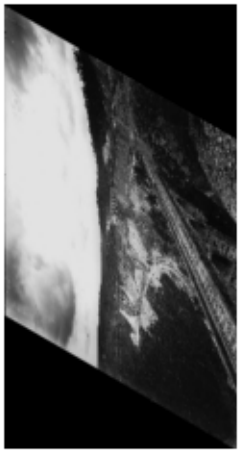
Figura 2.3: Rotar una imagen respecto a su centro



(a) Escalado uniforme (1/4)



(b) Shear horizontal (-30º)



(c) Rotación antihoraria (90ª)

Figura 2.4: Composición de transformaciones afines sobre una imagen

2.2.2. Implementación de las transformaciones compuestas

El resultado de realizar las transformaciones mediante composición de matrices, se muestra en la Figura 2.5a, geométricamente el resultado es el mismo. Sin embargo, si observamos el detalle de las intensidades de los píxeles resultantes, en la figura Figura 2.5b, no son iguales. Cada transformación sobre la imagen realiza una interpolación, en las transformaciones individuales por tanto se están aplicando tres interpolaciones, por una en el caso de la transformación compuesta.

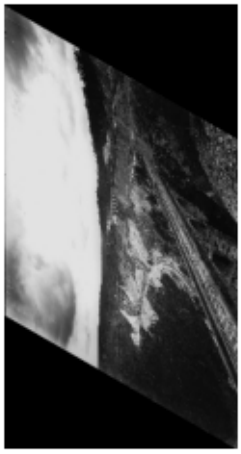
Al realizar transformaciones geométricas de 2D sobre una imagen, las coordenadas de los píxeles de la imagen resultante no tienen por qué coincidir con las de la imagen original.

Para determinar el valor de cada píxel de la imagen de destino, OpenCV utiliza un proceso de **mapeo inverso**, que consiste en recorrer los píxeles de la salida y calcular sus coordenadas correspondientes en la imagen de origen [5].

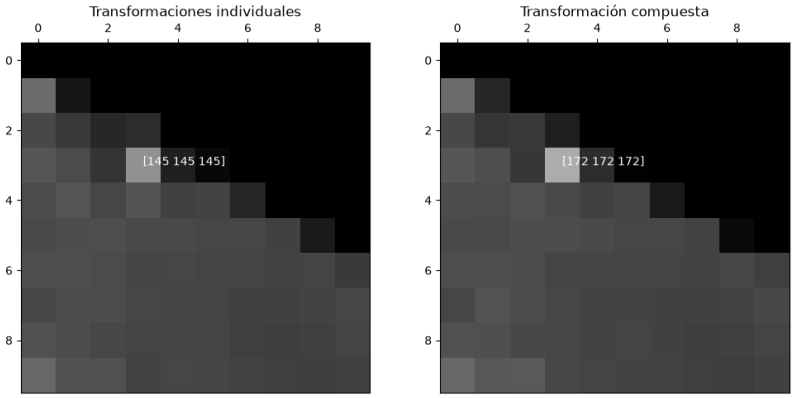
$$\mathrm{dst}(x, y) = \mathrm{src}\left(f_x(x, y), f_y(x, y)\right)$$

donde $f_x(x, y)$ y $f_y(x, y)$ representan las coordenadas de la imagen original correspondientes al píxel (x, y) de la imagen destino.

Estas coordenadas toman valores decimales y, por tanto, no tienen porque tener una correspondencia exacta con un la posición de un píxel de la imagen original. Para estimar la intensidad del píxel de la imagen de salida se aplican diferentes métodos de interpolación.



(a) Resultado de la transformación



(b) Detalle de las imágenes transformadas

Figura 2.5: Aplicación de transformaciones afines mediante composición de matrices

La interpolación por el vecino más cercano consiste en asignar a cada píxel de destino el valor del píxel de origen cuya posición sea la más cercana a las coordenada calculadas [6]. Como se muestra en Figura 2.6, el valor del punto interpolado se obtiene directamente del píxel más cercano.

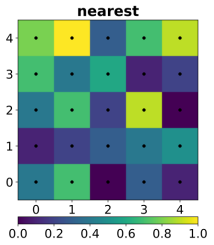


Figura 2.6: Interpolación 2D del vecino más cercano

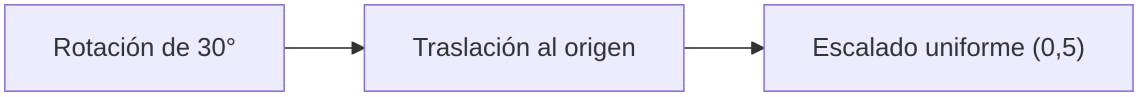
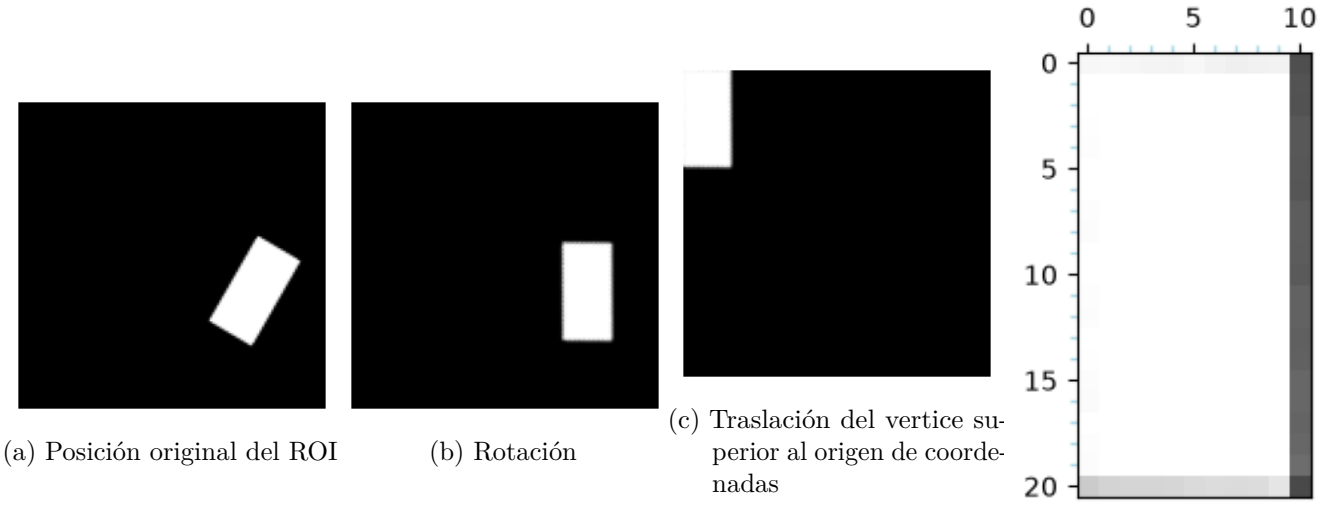


Figura 2.7: Secuencia de transformaciones geométricas



(a) Posición original del ROI

(b) Rotación

(c) Traslación del vertice superior al origen de coordenadas

(d) Escalado de la imagen

Figura 2.8: Secuencia de transformaciones para recolocar la región de interés

Este método de interpolación es computacionalmente poco costoso, no tiene que realizar promedios entre píxeles cercanos, y especialmente útil cuando es importante conservar los valores orginales de los píxeles, por ejemplo si vamos a aplicar máscaras de segmentación es pasos posteriores.

2.2.3. Transformar la región de interés (ROI)

Como se ilustra en Figura 2.8, el procedimiento consta de tres operaciones: primero se aplica una rotación de 30º en sentido antihorario para alinear la región con los ejes de coordenadas, luego se realiza una traslación para situar el vertice superior izquierdo en el origen y finalmente se aplica un escalado uniforme para reducir sus dimensiones a la mitad.

2.3. Ejercicio 2.3

2.3.1. Deformación de una imagen

Como se observa en Figura 2.9, la transformación modifica únicamente las coordenadas horizontales, manteniendo las verticales sin cambios:

- Mitad izquierda: se aplica un factor de escala de $2/3$, comprimiendo su anchura original de $W/2$ a $W/3$.
- Mitad derecha: se aplica un factor de escala de $4/3$, expandiendo su anchura original de $W/2$ a $2W/3$.

Finalmente, ambas regiones se combinan para obtener la imagen transformada, conservando la anchura total original W y manteniendo la continuidad en el punto de unión entre las dos mitades.



(a) Mitad izquierda: escala 2/3



(b) Mitad derecha: escala 4/3



(c) Imagen resultante

Figura 2.9: Deformación horizontal mediante una transformación lineal por tramos

2.3.2. Transformación no lineal

Se considera la siguiente transformación geométrica:

$$\begin{aligned} y' &= y + 0.4(x - 0.5)^2 - 0.1 \\ x' &= 2x \end{aligned} \tag{2.1}$$

La transformación Ecuación 2.1 es **no-lineal**, debido a la dependecia cuadrada de la coordenada x, a diferencia de las trasnformaciones afines esta transformación no conserva las líneas rectas, como se observa en la figura Figura 2.10.

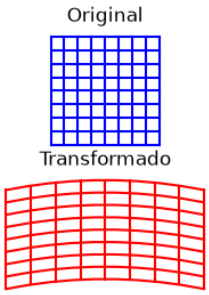


Figura 2.10: Transformación no lineal

Aplicamos la tranformación Ecuación 2.1 a la imagen siguiendo la secuencia:



Figura 2.11: Secuencia de transformaciones no lineales

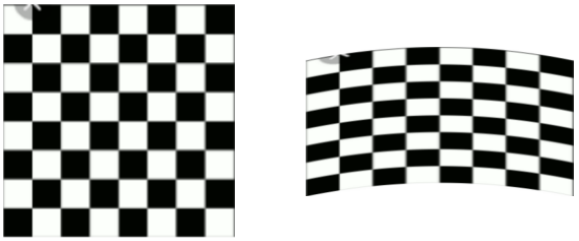


Figura 2.12: Transformación no lineal del tablero de ajedrez

3 Calibración de la cámara

3.1. Ejercicio 3.1

Para la captura de la imágenes he implementado un pequeño programa CLI que puede ejecutar con el comando `uv run miia-pec1 capture`, que permite visualizar en tiempo real las esquinas detectadas para facilitar la captura del tablero en diferentes posiciones y orientaciones generando imágenes válidas para la calibración.

En Figura 3.1 se muestran las imágenes seleccionadas para el proceso de calibración.

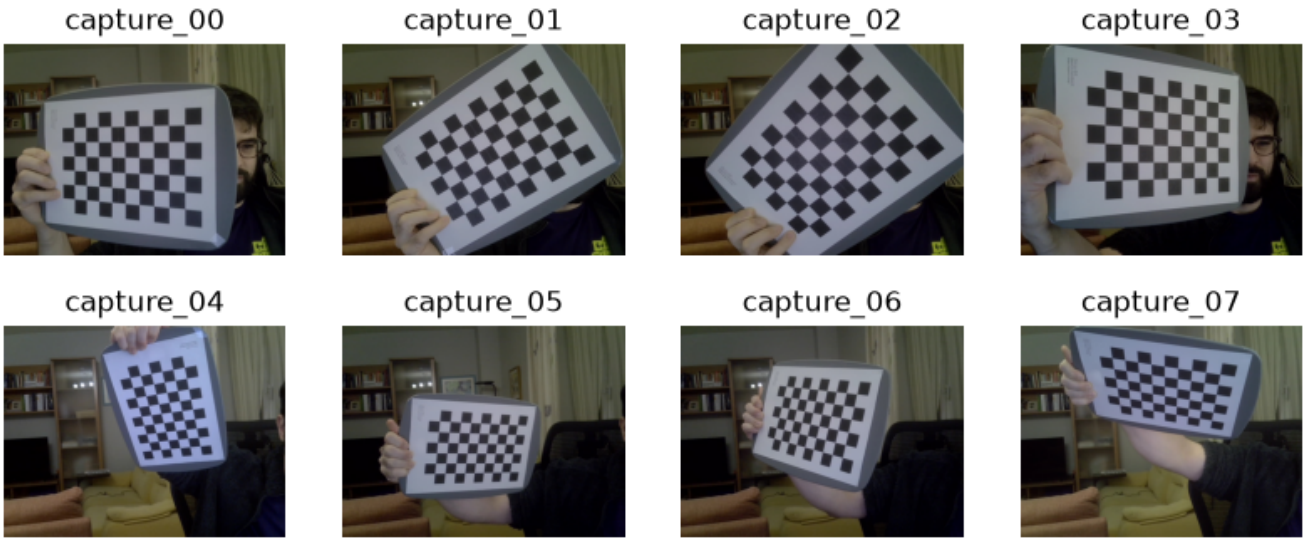


Figura 3.1: Imágenes del patrón de ajedrez utilizadas para la calibración

Posteriormente, el comando `uv run miia-pec calibrate` realiza la calibración de la cámara. A partir de las correspondencias entre los puntos del patrón y sus proyecciones en las imágenes, estima los parámetros de la cámara y los coeficientes de distorsión.

Como se observa en Figura 3.2, el algoritmo identifica las esquinas interiores del tablero de ajedrez, que se utilizan como puntos de referencia para calcular los parámetros de calibración.

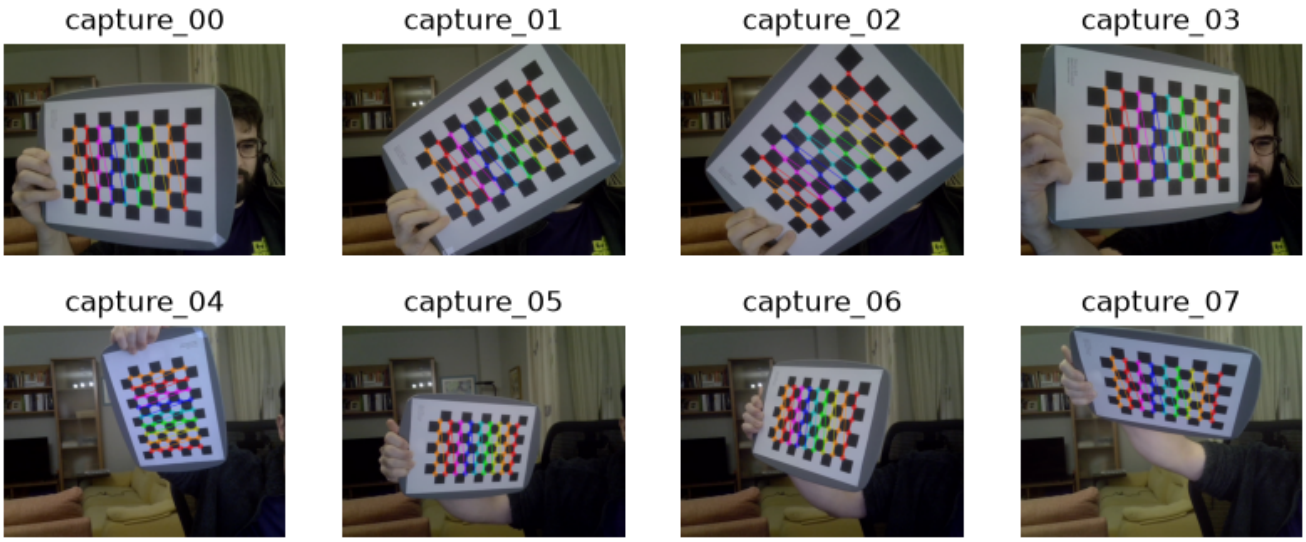


Figura 3.2: Detección de esquinas interiores del patrón de calibración

En Tabla 3.1 se muestran los parámetros obtenidos durante la calibración, la matriz intrínseca **K** y los coeficientes de distorsión **D**

Estos parámetros permiten corregir la distorsión de las imágenes y realizar proyecciones geométricas más precisas.

Tabla 3.1: Resultados de la calibración de la cámara

Parámetro	Valor
Matriz intrínseca K	$\begin{bmatrix} 701.7948 & 0.0000 & 316.8115 \\ 0.0000 & 702.0579 & 236.2497 \\ 0.0000 & 0.0000 & 1.0000 \end{bmatrix}$
Distorsión D	$(0.0183, 0.6575, -0.0077, 0.0025, -1.8116)$

3.1.1. Evaluación del error de calibración

En Figura 3.3 se puede observar el error de calibración para cada imagen, junto con el error de calibración global RMS.

Los resultados muestran errores relativamente reducidos, con pequeñas diferencias entre las imágenes utilizadas. El error RMS global es aproximadamente de 0,276 píxeles, lo que indica una buena correspondencia entre las observaciones y el model estimado.

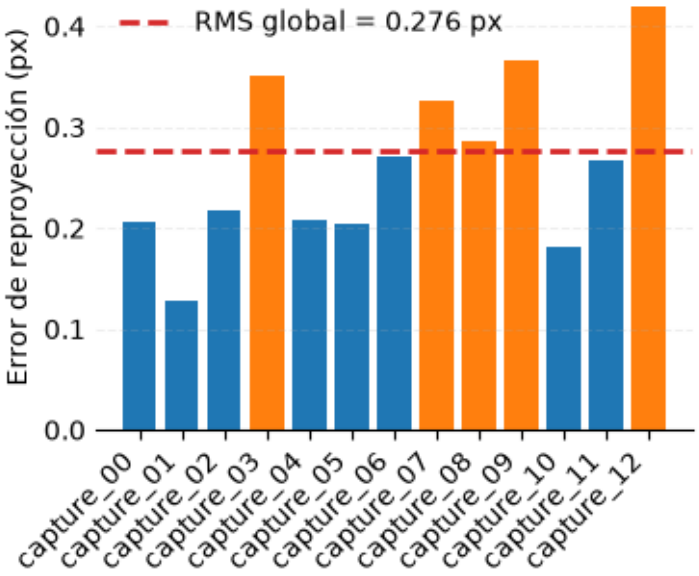


Figura 3.3: Error de reproyección por imagen y RMS global

En Figura 3.4 se compara la imagen original con su versión corregida. Aunque las diferencias son sutiles, se aprecia una ligera rectificación de las líneas y los bordes, especialmente en las zonas periféricas de la imagen.



(a) Imagen original

(b) Imagen corregida

Figura 3.4: Comparación de la imagen original y la imagen corregida mediante calibración

3.2. Ejercicio 3.2

Partimos de unos puntos 3D conocidos del objeto y detectamos sus correspondientes posiciones 2D en la imagen.

Tabla 3.2: Correspondencias entre los puntos 3D del tablero y sus posiciones 2D detectadas en la imagen

Puntos 3D conocidos	Posiciones 2D en la imagen
$(0.0000, 0.0000, 0.0000)$	$(380.1252, 332.5765)$

Puntos 3D conocidos	Posiciones 2D en la imagen
$(1.0000, 0.0000, 0.0000)$	$(360.6099, 333.1312)$
$(2.0000, 0.0000, 0.0000)$	$(340.7096, 333.4769)$
$(3.0000, 0.0000, 0.0000)$	$(320.2885, 333.8613)$
$(4.0000, 0.0000, 0.0000)$	$(299.3304, 334.4000)$



(a) Proyección de los ejes de coordenadas del tablero

(b) Proyección de un cubo virtual sobre el tablero

Figura 3.6: Estimación de la posición del tablero y visualización mediante elmentos virtuales 3D

Este procedimiento podría aplicarse a la cabeza de una persona, sería necesario un modelo facial 3D de referencia junto con el sistema capaz de detectar las posiciones 2D de puntos significativos. A partir de las correspondencias, `cv.solvePnP()` permitiría estimar la orientación de la cabeza respecto a la cámara, como resume la Figura 3.7.

Sin embargo, la orientación de la cabeza no tiene por qué coincidir con la orientación de la mirada, ya que los ojos tienen movilidad independiente. Para estimar la mirada con mayor precisión es necesario detectar los iris y estimar la orientación de los ojos respecto a la cabeza.

Finalmente, combinando la rotación de la cabeza con la dirección ocular estimada, se podría obtener una aproximación de la dirección tridimensional de la mirada respecto a la cámara.

A partir de las correspondencias entre los puntos 3D y 2D, la matriz intrínseca y los coeficientes de distorsión de la cámara, `cv.solvePnP()` permite estimar la rotación y la traslación del objeto respecto a la cámara. Estos parámetros se calculan buscando que las proyecciones de los puntos 3D se ajusten lo mejor posible a los puntos 2D detectados. Esta transformación se ilustra en la Figura 3.5.

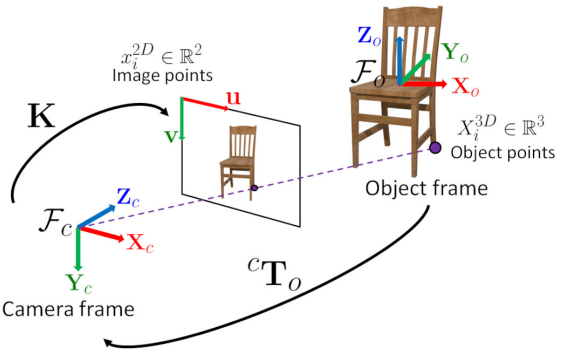


Figura 3.5: Sistemas de coordenadas de la cámara y del objeto y proyección de puntos 3D en el plano de imagen. Fuente: OpenCV [7].

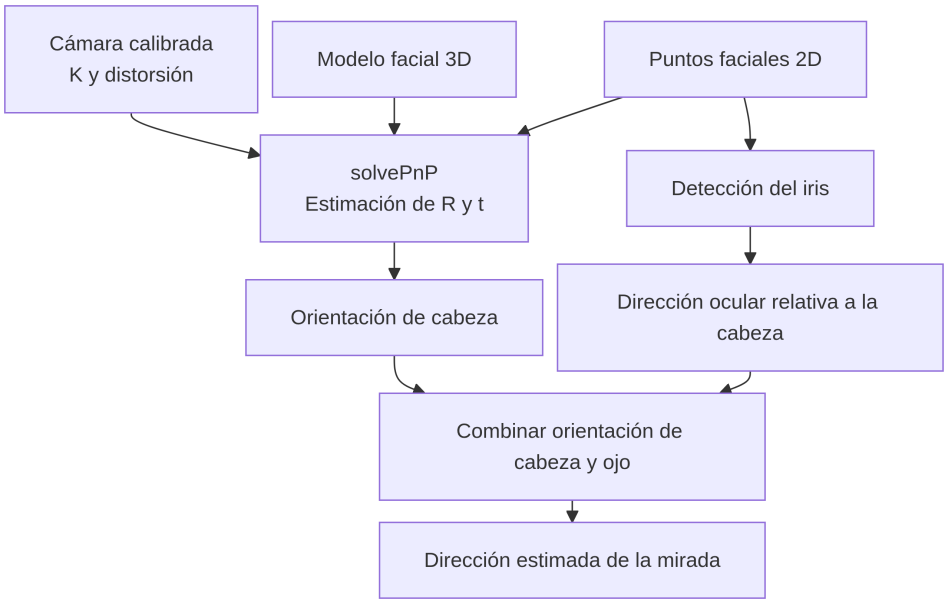


Figura 3.7: Proceso para estimar la orientación de la cabeza y la dirección de la mirada.

4 Flujo óptico y seguimiento

4.1. Ejercicio 4.1

4.1.1. Lucas-kanade optical flow

En la Figura 4.1 se muestra el desplazamiento estimado de varios puntos característicos de un vehículo entre dos fotogramas consecutivos. Los puntos verdes representan las posiciones anteriores y los azules las posiciones estimadas en el nuevo fotograma.

El seguimiento sucesivo de estos puntos permite reconstruir sus trayectoria a lo largo del vídeo, como se observa en la Figura 4.2.

4.1.2. Dense optical flow

El algoritmo Farnebäck estima el vector de desplazamiento (u, v) que satisface aproximadamente la relación:

$$I_1(x + u,\ y + v) \approx I_0(x, y)$$

donde I_0 e I_1 representan dos fotogramas consecutivos de una secuencia de vídeo.

En la Figura 4.3 se muestran dos fotogramas consecutivos de la secuencia analizada, en ella se observa el desplazamiento de varios peatones.

En la Figura 4.4 se muestran tres formas de representar el flujo óptico estimado: - **(a) Representación vectorial:** se visualizan los deplazamientos mediante segmentos dibujados sobre una malla representada sobre la imagen. Su orientación indica la dirección del movimiento y su longitud la magnitud del desplazamiento. - **(b) Representación HSV:** se codifica el flujo óptico mediante colores, el tono representa la dirección del movimiento y el brillo su magnitud normalizada, las regiones con mayor desplazamiento aparecen más brillantes. - **(c) Deformación de la imagen:** se utiliza el campo de de flujo optico para remuestrear la imagen aplicando un mapeo inverso, para visualizar el efecto estimado del desplazamiento en la imagen.



Figura 4.1: Desplazamiento de puntos característicos entre dos fotogramas mediante Lucas-Kanade

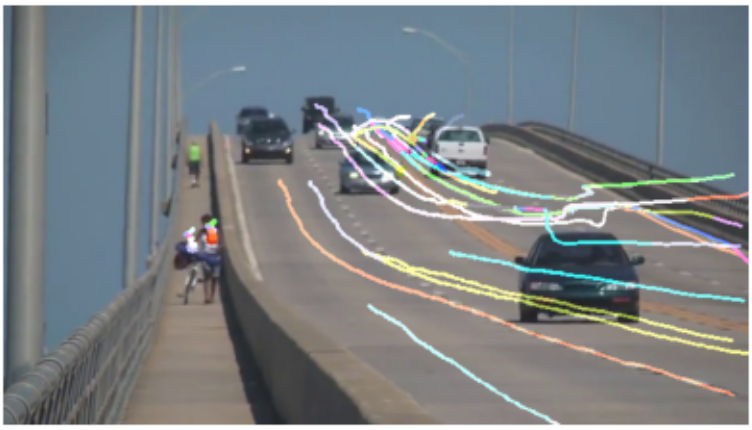
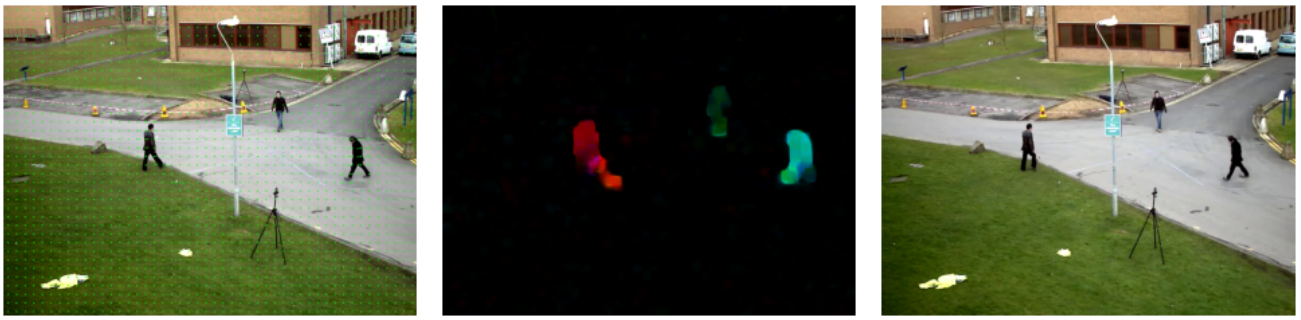


Figura 4.2: Trayectorias de puntos característicos durante el seguimiento óptico



Figura 4.3: Fotogramas consecutivos utilizados para estimar el flujo óptico denso



(a) Vectores de desplazamiento muestreados regularmente

(b) Representación HSV de la dirección y magnitud del movimiento

(c) Deformación de la imagen mediante mapeo inverso a partir del campo de flujo óptico

Figura 4.4: Visualización del flujo óptico denso estimado mediante Farnebäck



(a) Imagen sin registrar (b) Secuencia registrada (c) Objetivo

Figura 4.5: Visualización flujo óptico

Otra forma de visualizar el flujo óptico consiste en construir una imagen RGB, asignando la imagen transformada mediante el campo de flujo al canal rojo y la imagen de referencia a los canales verde y azul.

Cuando ambas imágenes coinciden, el resultado aparece en escala de grises, mientras que las diferencias se manifiestan mediante tonalidades rojizas o cian, permitiendo identificar posibles errores de registro.

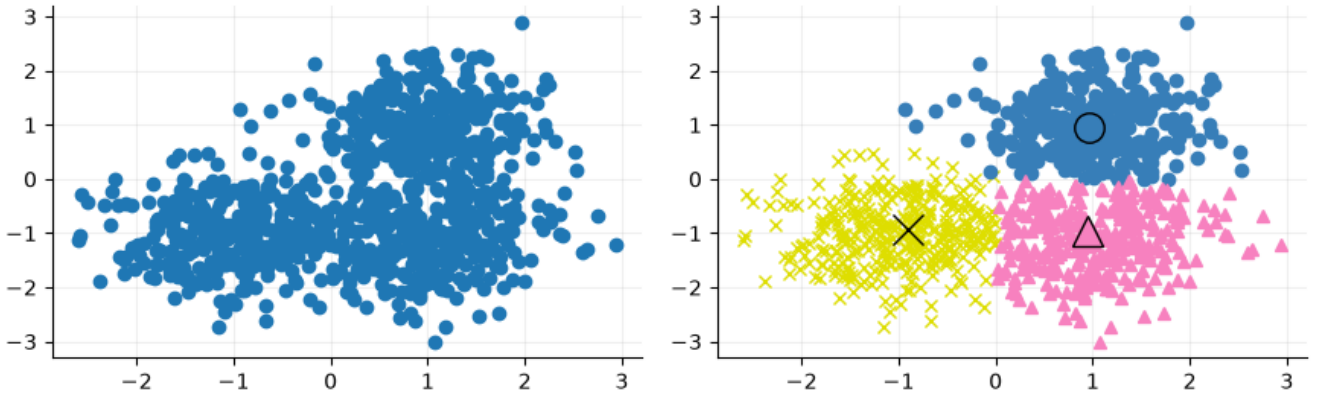
En la Figura 4.5 se observa cómo la compensación del movimiento mediante Farnebäck reduce las diferencias entre los fotogramas.

4.2. Ejercicio 4.2

El algoritmo de **Mean Shift**, sitúa una ventana alrededor de un punto inicial, calcula la media de los puntos contenidos en ella y desplaza la ventana a esa posición, repitiendo el proceso hasta que el desplazamiento es prácticamente nulos permitiéndonos localizar zonas de alta densidad como se muestra en la Figura 4.6.

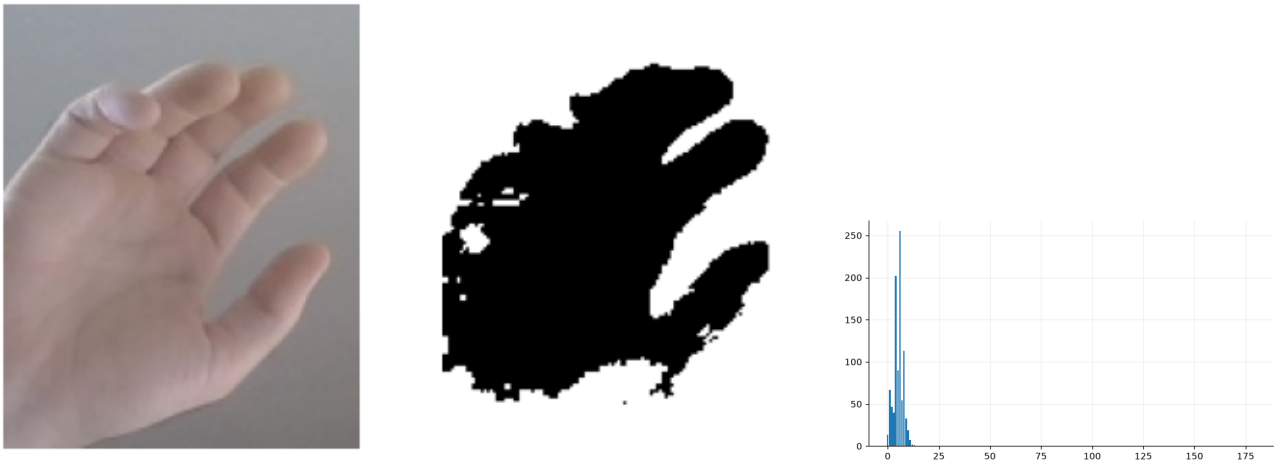
En la Figura 4.7 se muestra la región de interés, la máscara obtenida y el histograma normalizado que caracteriza los tonos de la mano.

Con el histograma se calcula la retroproyección, utilizando `cv.calcBackProject()`, funciona asignado a cada píxel de la imagen un valor según la correspondencia de su tono, con los tonos normalizados del histograma. De esta forma, se genera una imagen en escala de grises donde los tonos más similares a los de la mano se presentan más oscuros.



(a) Distribución inicial de los puntos (b) Agrupaciones y centros detectados mediante Mean Shift

Figura 4.6: Identificación de regiones de alta densidad mediante Mean Shift



(a) Región de interés (ROI) de la mano (b) Máscara binaria obtenioda mediante umbrales HSV (c) Histograma normalizado del canal H

Figura 4.7: Obtención del modelo de color de la mano mediante segmentación HSV e histograma normalizado

En la Figura 4.8a se muestra la retroproyección obtenida con la imagen de referencia. En la Figura 4.8b se representan las agrupaciones detectadas al aplicar MeanShift sobre las coordenadas de los píxeles seleleccionados, podemos observar que, además de la mano aparecen otras regiones similares, especialmente alrededor de la cara.

Para realizar el seguimiento en tiempo real utiliza el comando `uv run miia-pec1 meanshift`.

Algunos consideraciones a tener en cuenta:

- Necesita una ventana inicial para realizar el seguimiento.
- Para que el seguimiento sea correcto es necesario que las condiciones de iluminación sean similares a los de la captura de referencia.
- El tamaño de la ventana es fijo, no se adapta aunque cambie la escala de la mano.
- Se pierde el foco al salir de plano, se realiza un movimiento brusco hacia la ventana inicial o zonas de tonalidades similares a la mano como la cara.

```
# Este comando realiza el seguimiento en la webcam de un objeto usando meanshift
# Recibe una imagen de referencia, el área de interés y los valores mínimo y máximo con
 ↪  los que generar la máscara para el seguimiento.
!uv run miia-pec1 meanshift \
    --track-window 210,110,120,150 \
    --mask-min 0,35,70 \
    --mask-max 20,130,255 \
    --image-ref {OUT_DIR / 'seguimiento.jpg'} \
    --device 2 \
    --outfile {OUT_DIR / 'ejemplo.mp4'}
```



(a) Retroproyección del histograma de color

(b) Agrupaciones espaciales identificadas mediante MeanShift

Figura 4.8: Identificación de regiones con colores similares a los de la mano mediante retroproyección y agrupamiento MeanShift

5 Segmentación sin conocimiento del dominio

5.1. Umbralización

5.1.1. Umbralización global

La umbralización global utiliza un único umbral para segmentar toda la imagen.

$$dst(x,y) = \begin{cases} valor_x, & \text{si } src(x,y) > thresh \\ valor_y, & \text{si } src(x,y) \leq thresh \end{cases}$$

Donde:

- $valor_x$: valor asignado cuando se supera el umbral.
- $valor_y$: valor asignado cuando no se supera el umbral.

Los histogramas de la Figura 5.1 permiten analizar cómo se modifica la distribución de intensidades con cada método. Mientras las umbralizaciones binarias concentran los píxeles en dos valores, los métodos de truncamiento y puesta a cero conservan parte de las intensidades originales.

En la Figura 5.2 se puede observar que los resultados son dependientes del contraste y la iluminación de cada imagen.

5.1.2. Umbralización local

La umbralización local, calcula un umbral $T(x,y)$ para cada píxel a partir de una ventana centrada en él.

En la Figura 5.3 se muestra como la umbralización local, permite recuperar una mayor cantidad de detalles que la umbralización global sobre imágenes que presentan variaciones en la iluminación, ya permite ajustar el umbral de cada zona, las zonas iluminadas tendrán un umbral alto mientras que las oscuras presentarán uno más bajo.

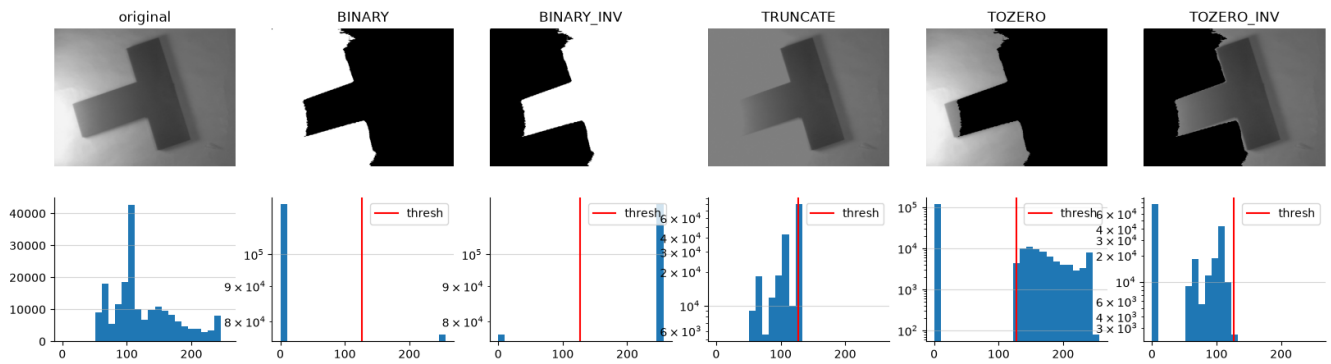


Figura 5.1: Distribución de intensidades tras aplicar los distintos métodos de umbralización global

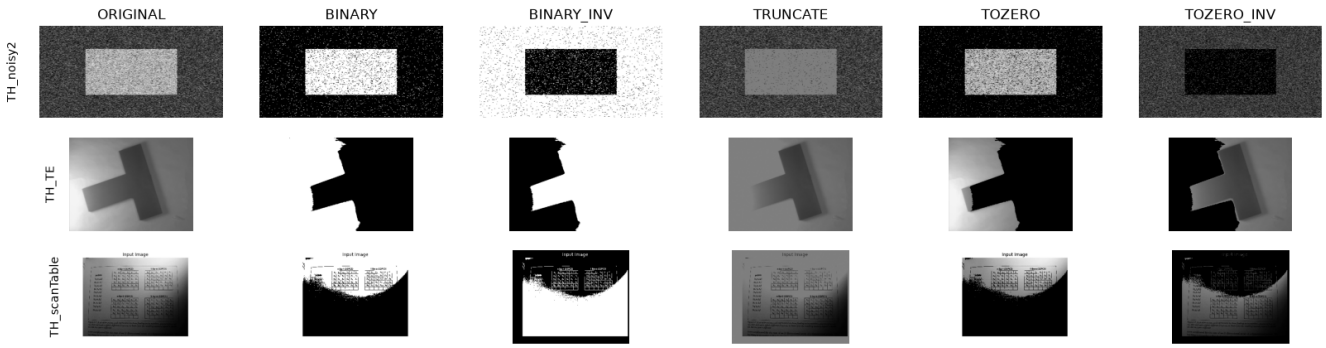


Figura 5.2: Comparación de los métodos de umbralización global (umbral = 127)

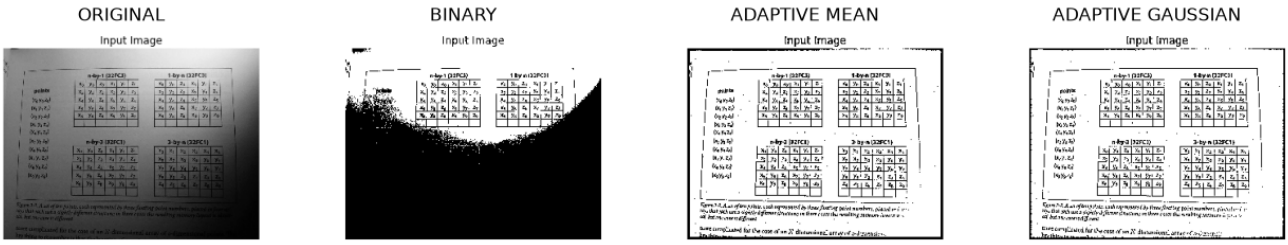


Figura 5.3: Umbralización global vs umbralización local

En la Figura 5.4 se observa como al aumentar el valor de C se reduce la presencia de píxeles negros aislados en el fondo, mientras que el valor de `blockSize` determina el tamaño de la la extensión que se utiliza para calcular el umbral.

Para analizar el efecto del tamaño de la ventana, sobre el resultado, se selecciona un recorte que contiene parte del rectángulo y del fondo.

En la Figura 5.5, se observa como una ventana más pequeña, produce una segmentación más fragmentada, dado que el umbral se calcula a partir de una región reducida de píxeles vecinos que tienen intensidades similares al píxel central de la ventana. Al aumentar el tamaño de la ventana está incorpora píxeles de las dos regiones del recorte lo que favorece su diferenciación asemejando su comportamiento a una umbralización global.

Como se muestra en las figuras Figura 5.6 y Figura 5.7, es posible obtener una segmentación bastante completa utilizando tamaños de ventana grandes, para que las ventanas incluyan píxeles del objeto que estamos buscando segmentar y del fondo.

En el caso del rectángulo se obtiene un buen resultado con `blockSize=511` y $C = 10$, mientras que para la pieza en forma de T el mejor resultado se obtiene con `blockSize=501` y $C = 20$.

Sin embargo, ninguna de las configuraciones consigue una segmentación completa y sin errores, habría que considerar otras técnicas.

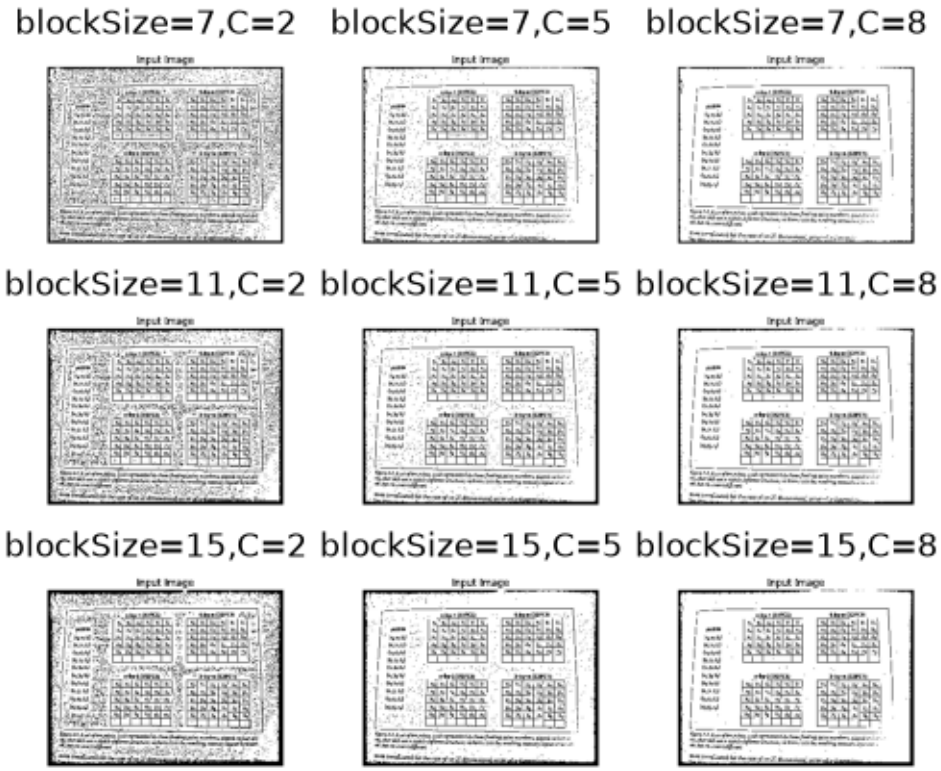
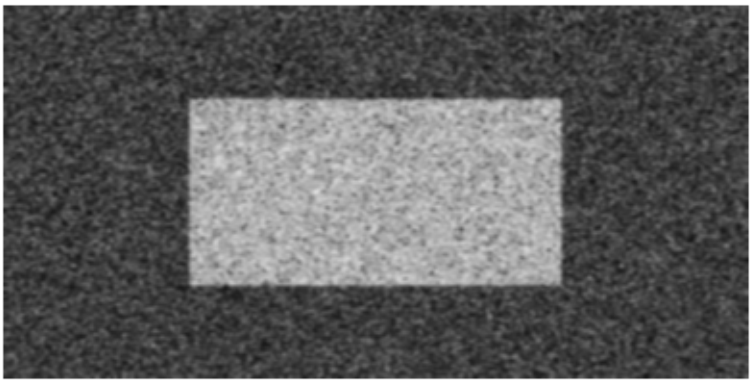


Figura 5.4: Ajuste de parámetros para la umbralización local

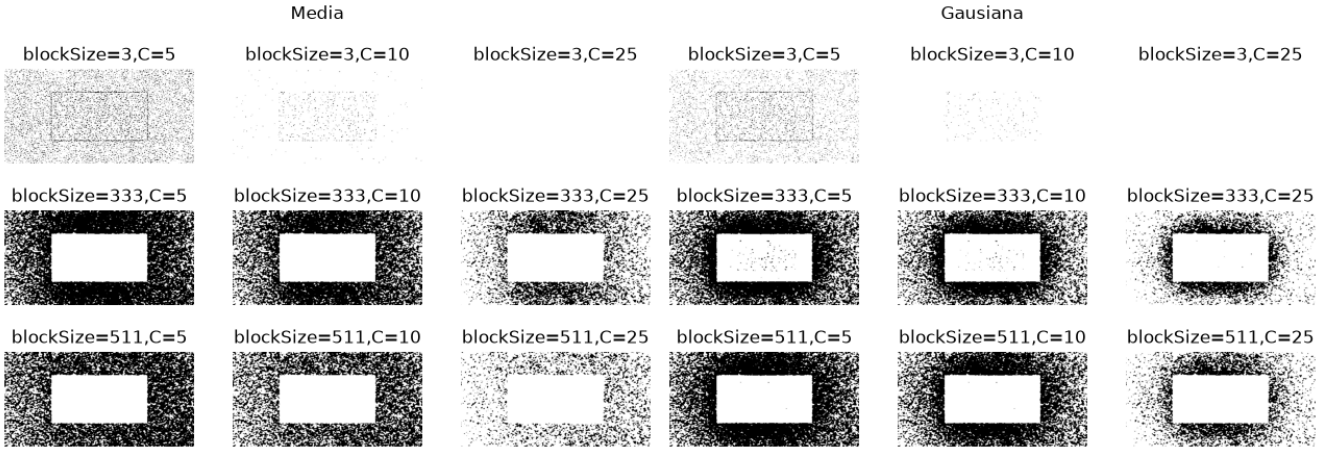


(a) Recorte de una región (b) Umbralización con una ventana pequeña (blockSize=15) (c) Umbralización con una ventana grande (blockSize=81)

Figura 5.5: Análisis de la umbralización local en regiones homogéneas

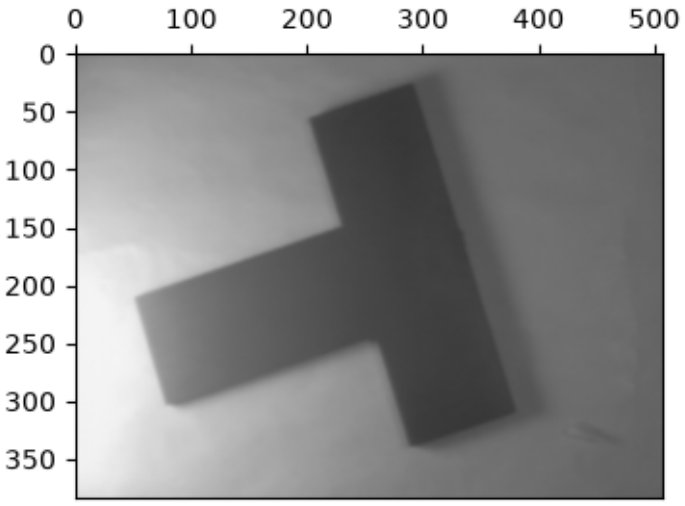


(a) Imagen original

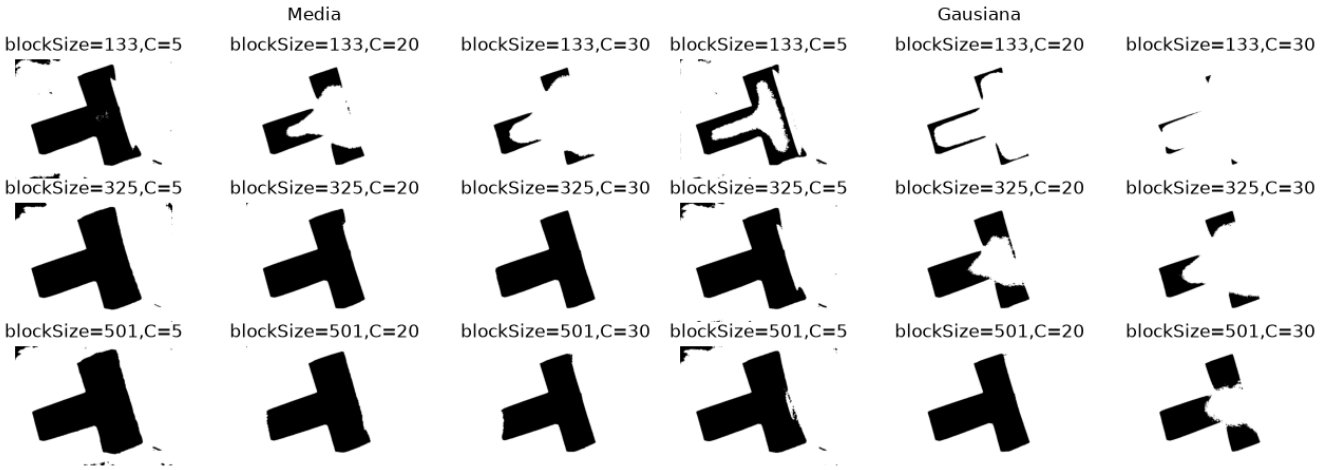


(b) Aplicación de umbralización local, usando la media (c) Aplicación de umbralización local, usando la gausiana

Figura 5.6: Ajuste de parámetros para la umbralización local



(a) Imagen original



(b) Aplicación de umbralización local, usando la media (c) Aplicación de umbralización local, usando la gausiana

Figura 5.7: Ajuste de parámetros para la umbralización local

5.2. K-means clustering

La Figura 5.8 muestra la imagen original con su histograma de intensidades, se observa una concentración elevada en las intensidades bajas, asociadas a las regiones oscuras y al fondo de la imagen.

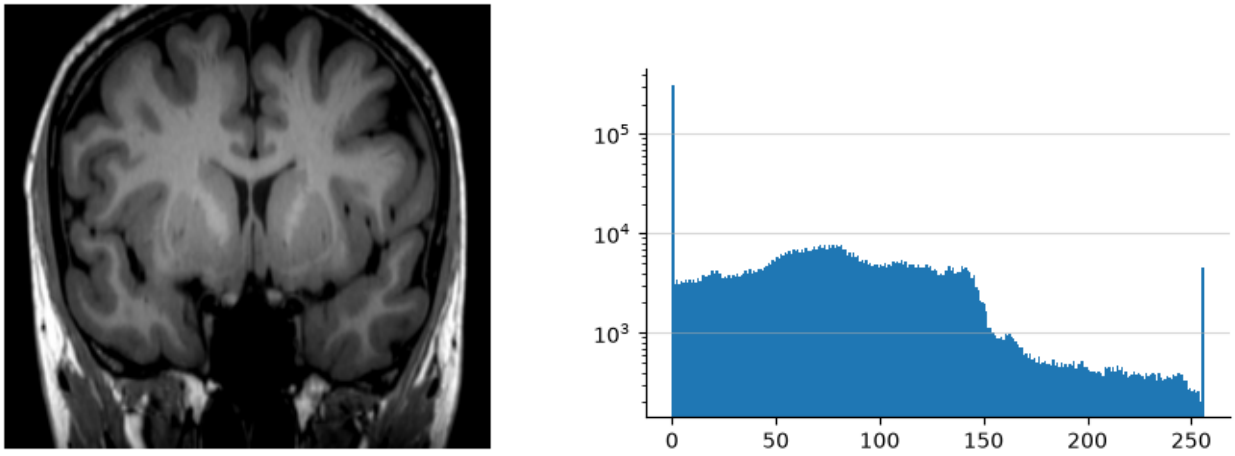
La Figura 5.9 muestra los resultados de aplicar el K-means con diferente número de clústeres. Para cada configuración se muestra la imagen segmentada junto con el histograma de intensidades original, coloreado según los grupos obtenidos, las líneas verticales naranjas indican la posición de los centroides calculados por el algoritmo.

Para valores reducidos de k, como k=2 y k=3, la segmentación permite diferenciar regiones más oscuras de las estructuras del cerebro, pero se pierde el detalle anatómico.

Al aumentar el número de clusters k=5 y k=7, presenta una mayor variedad de intensidades intermedias y altas. Esto permite visualizar más estructuras del tejido cerebral.

Para k=12 y k=15 se incorporan nuevos centroides que mejoran la representación de las transiciones entre niveles de gris permitiendo distinguir más detalles de la imagen.

Los resultados muestran que aumentar el número de clústeres mejora la fidelidad de la imagen segmentada pero aumenta la complejidad de la imagen.



(a) Imagen original

(b) Histograma de intensidades de la imagen original

Figura 5.8: Resonancia magnética cerebral

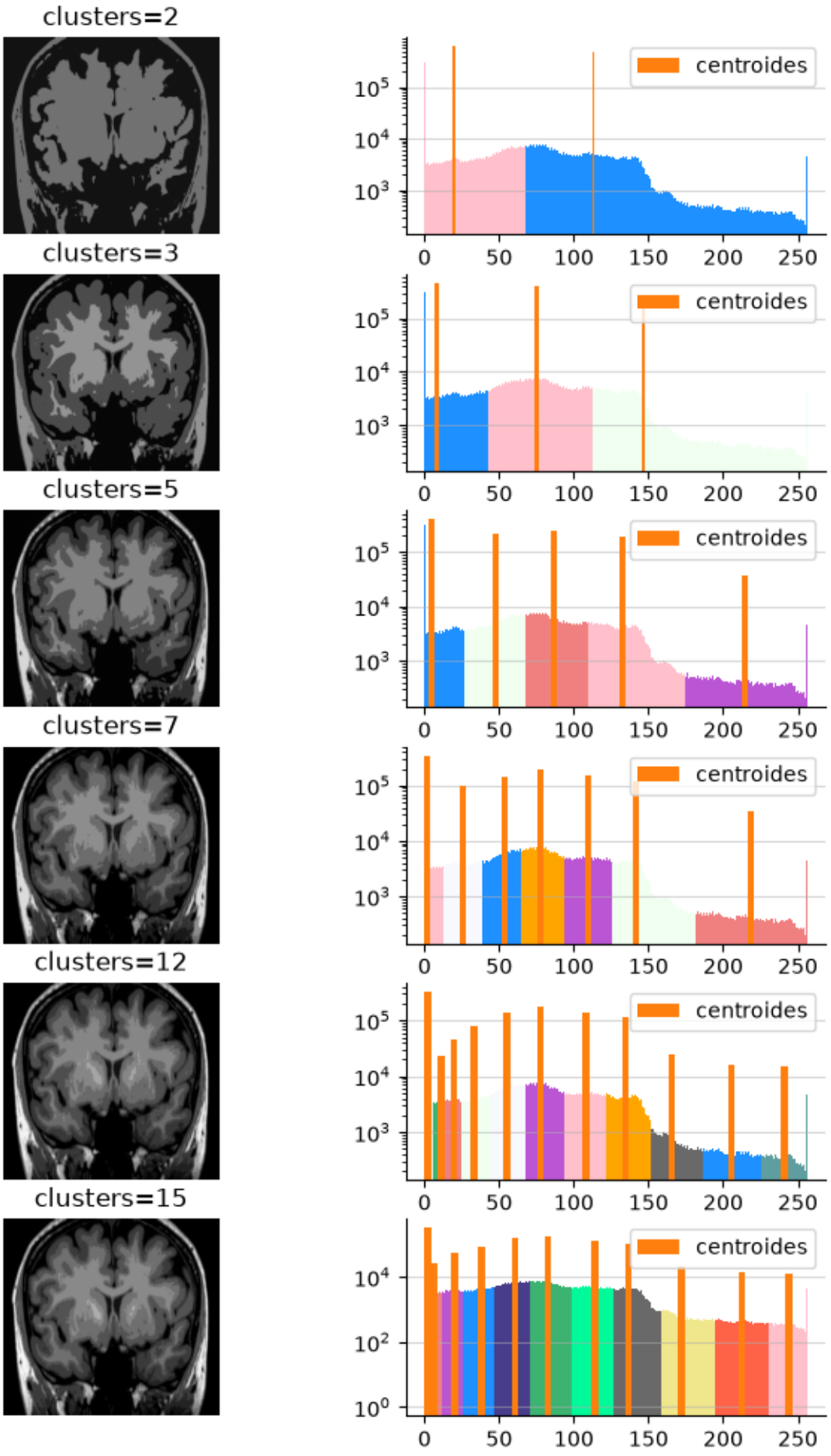


Figura 5.9: Segmentación mediante K-means para distintos números de clústeres

5.3. Segmetación con MeanShift

El algoritmo Mean Shift que nos permite suavizar una imagen conservando los bordes. OpenCV proporciona una implementación de este algoritmo, funciona desplazando de forma iterativa una ventana de búsqueda hacia regiones de mayor densidad en el espacio conjunto de espacio y color. Para ello, se consideran los píxeles que cumplen las siguientes condiciones:

$$\left\{ (x,y) : \begin{array}{c} X - \texttt{sp} \leq x \leq X + \texttt{sp}, \\ Y - \texttt{sp} \leq y \leq Y + \texttt{sp}, \\ \|(R,G,B) - (r,g,b)\|_2 \leq \texttt{sr} \end{array} \right\}$$

Donde:

- **sp:** representa el radio de la ventana espacial y determina la distancia máxima considerada entre píxeles vecinos.
- **sr:** establece el radio de similitud cromática, limitando la diferencia de color entre los píxeles considerados.
- **(X,Y):** corresponde a la posición central de la ventana de búsqueda.
- **(R,G,B):** representa el color de referencia de la ventana.

Mediante este proceso, los píxeles con características espaciales y cromáticas similares convergen hacia valores representativos próximos, dando lugar a regiones de color más uniforme.

La Figura 5.10 presenta los resultados de aplicar Mean Shift con diferentes valores de los parámetros sp y sr, con el objetivo de analizar su influencia en el suavizado y la conservación de los detalles.

En la Figura 5.10a se aumenta el radio espacial, manteniedo el mismo radio de color. Al aumentar el radio espacial, el algoritmo considera vecinos píxeles más alejados favoreciendo la formación de regiones homogéneas más grandes, se reducen detalles pero las estructuras principales permanecen reconocibles.

En la Figura 5.10b mantiene el radio espacial y se incrementa el radio de color. El aumento del radio de color permite agrupar píxeles con diferencias de color mayores, lo que se traduce en un suavizado más intenso.







(a) Variación del radio espcial

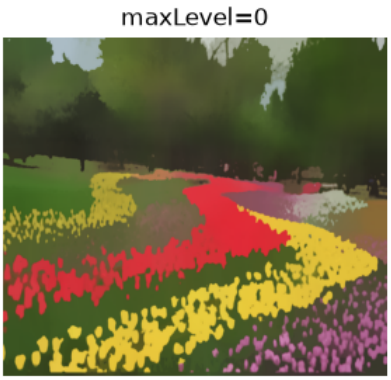


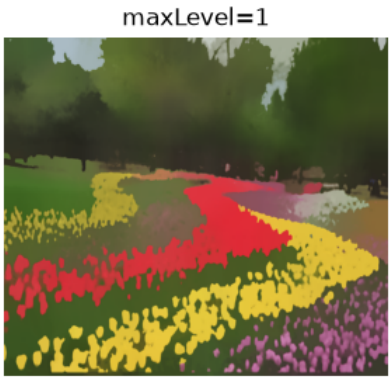




(b) variación del radio de color

Figura 5.10: Efecto de los parámetros en el filtrado Mean Shift





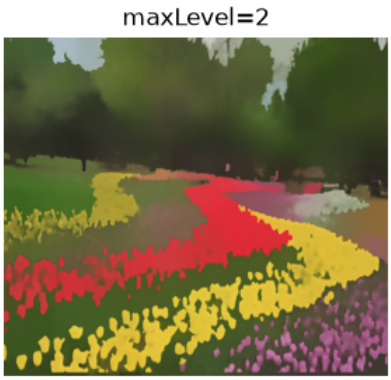


Figura 5.11: Resultado de pirámides gaussianas al aplicar mean shift

5.4. Segmentación Watershed

En la figura Figura 5.12 se muestra la segmentación de los lóbulos cerebrales utilizando un umbral de Otsu sobre la imagen suavizada y enmascarada con una elipse que representa la región del cerebro.

En la Figura 5.13a se muestran las transformaciones de fondo seguro, primer plano seguro y región desconocida que se utilizan para construir la máscara de regiones segmentadas, Figura 5.13b, que es utilizada como semilla por el algoritmo de watershed.

En la Figura 5.14 se muestra el resultado de la aplicación del algoritmo watershed para la separación de los lóbulos.

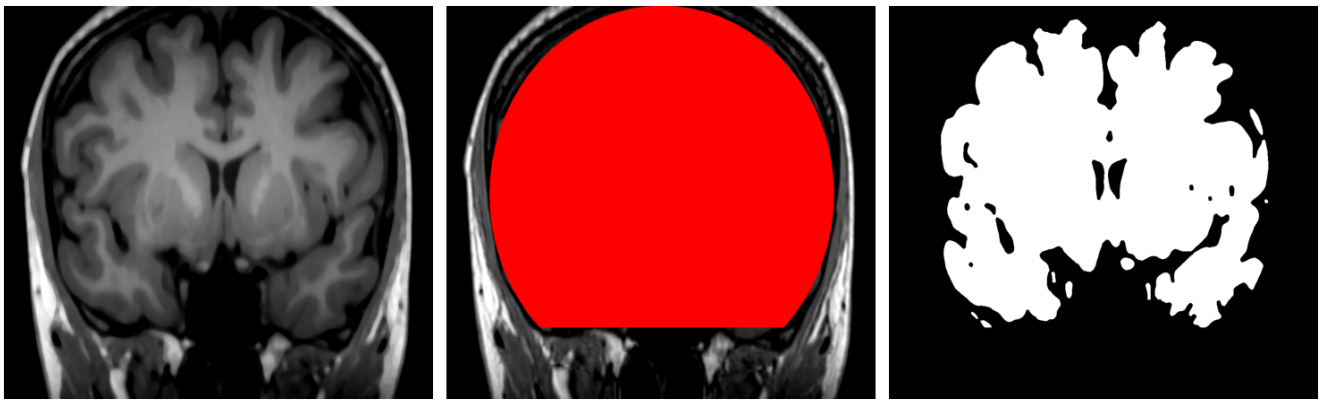


Figura 5.12: Segmentación de los lóbulos cerebrales



(a) Fondo seguro, primer plano seguro y región desconocida



(b) Máscara de las regiones segmentadas

Figura 5.13: Transformaciones intermedias

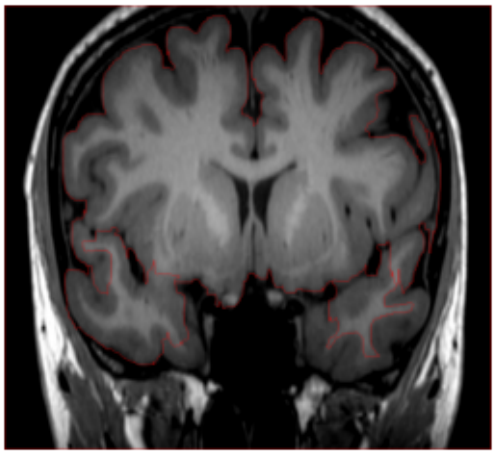


Figura 5.14: Separación de los lóbulos con watershed

Referencias

[1] Posit Software, PBC, *Quarto*. (2026). Disponible en: https://quarto.org

[2] Hugging Face, «Hugging Face: The AI Community Building the Future». https://huggingface.co/.

[3] P. Moracho, «Respuesta a: ¿Cómo usar contours apropiadamente?» Stack Overflow en español, 13 de abril de 2018. Disponible en: https://es.stackoverflow.com/a/155697/510086

[4] A. Rosebrock, «Rotate Images (Correctly) with OpenCV and Python». Accedido: 15 de octubre de 2026. [En línea]. Disponible en: https://pyimagesearch.com/2017/01/02/rotate-images-correctly-with-opencv-and-python/

[5] OpenCV, «Geometric Image Transformations». Accedido: 17 de octubre de 2026. [En línea]. Disponible en: https://docs.opencv.org/5.0/main_modules/imgproc_transform.html

[6] Wikipedia contributors, «Nearest-neighbor interpolation». Accedido: 17 de octubre de 2026. [En línea]. Disponible en: https://en.wikipedia.org/wiki/Nearest-neighbor_interpolation

[7] OpenCV, «3D vision functionality: solvePnP». Disponible en: https://docs.opencv.org/5.0/main_modules/d_projection.html#solvepnp